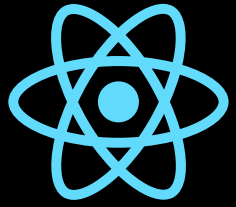


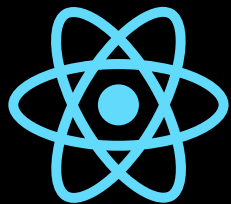
이벤트 핸들러란?

1. 웹페이지에서 **마우스 클릭, 키보드 입력** 등과 같은 **특정 사건**이 발생했을 때 **실행되는 코드 블록(함수)**을 의미한다.
2. 이벤트 핸들러를 특정 **DOM 요소 (버튼, 입력박스)**등에 **등록**하면 이벤트 발생을 기다렸다 원하는 기능을 처리한다.
3. **애플리케이션을 동적이고 상호작용**이 가능하도록 만드는 핵심적인 프로그래밍 요소이다.



이벤트 종류

1. 마우스 이벤트 : **onClick, onContextMenu**
2. 키보드 이벤트 : **onKeyDown, onChange**
3. 폼 이벤트 : **onSubmit, onReset**



이벤트 핸들러 등록

```
const [number, setNumber] = useState(0);
```

```
return(
```

```
  <div>
```

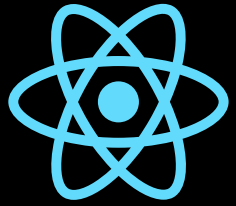
```
    <h1> {number} </h1>
```

```
    <button onClick={() => setNumber(number + 1)}>
```

```
      증가</button>
```

```
  </div>
```

```
)
```

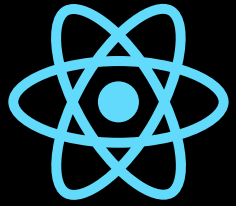


useRef

초기값을 useRef에 넣어주면 { **current: 초기값** } 객체가 반환된다.

```
const ref = useRef(초기값)
```

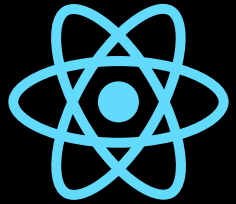
1. 값이 변경 되도 렌더링이 되지 않아 **불필요한 리 렌더링을 피한다.**
2. ID와 같은 **값을 보존**하거나 **DOM 요소에 접근**할 때 사용한다.
3. **ref.current**를 통해 접근 가능하다.



useRef 사용 예제 1

```
const idRef = useRef(1);  
return(  
  <div>  
    <h1> {idRef.current} </h1>  
    <button onClick={onRegister}>등록</button>  
  </div>  
)
```

```
const onRegister= () => {  
  console.log(idRef.current++)  
}
```



useRef 사용 예제2

```
const nameRef = useRef(null);  
  
return(  
  <form onSubmit={onSubmit}>  
    <input ref = {nameRef} />  
    <button type=submit>등록</button>  
  </form>  
)  
  
const onSubmit= (e) => {  
  e.preventDefault();  
  nameRef.current.focus();  
}
```